

```
// OSSP Skill 24 - Final Demonstration / Deliverable
// Demonstrates a small integrated Unix-shell-style program:
// commands, pipelines, redirection, background execution,
// signals, job tracking, and clean exit.
//
// This is a compact final demo suitable for presentation.
```

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <signal.h>
#include <sys/wait.h>
```

```
#define MAX_ARGS 32
#define MAX_JOBS 16
```

```
typedef struct {
    int id;
    pid_t pid;
    int active;
} Job;
```

```
static Job jobs[MAX_JOBS];
static int next_job = 1;
static volatile sig_atomic_t fg_pid = -1;
```

```
static void sigint_handler(int sig) {
    (void)sig;

    if (fg_pid > 0)
        kill(fg_pid, SIGINT);
    else
        write(STDOUT_FILENO, "\nUse 'exit' to quit.\n", 21);
}
```

```
static void add_job(pid_t pid) {
    for (int i = 0; i < MAX_JOBS; i++) {
        if (!jobs[i].active) {
```

```

        jobs[i] = (Job){next_job++, pid, 1};
        printf("[%d] %d\n", jobs[i].id, pid);
        return;
    }
}
printf("Job table full.\n");
}

static void update_jobs(void) {
    for (int i = 0; i < MAX_JOBS; i++) {
        if (!jobs[i].active)
            continue;

        int status;
        pid_t r = waitpid(jobs[i].pid, &status, WNOHANG);

        if (r == jobs[i].pid) {
            printf("[%d] Done\n", jobs[i].id);
            jobs[i].active = 0;
        }
    }
}

static int tokenize(char *line, char **args) {
    int n = 0;
    char *tok = strtok(line, " \t");

    while (tok && n < MAX_ARGS - 1) {
        args[n++] = tok;
        tok = strtok(NULL, " \t");
    }

    args[n] = NULL;
    return n;
}

static void run_simple(char **args, int background) {
    pid_t pid = fork();

```



sai_pavani@Sai-Pavani: ~/OS_



GNU nano 8.7.1

skill_24.c.c

```
if (pid == -1) {
    perror("fork");
    return;
}

if (pid == 0) {
    signal(SIGINT, SIG_DFL);
    execvp(args[0], args);
    perror("execvp");
    _exit(127);
}

if (background) {
    add_job(pid);
} else {
    fg_pid = pid;
    waitpid(pid, NULL, 0);
    fg_pid = -1;
}
}

static void run_pipeline(char *line) {
    char *parts[8];
    int count = 0;

    char *p = strtok(line, "|");
    while (p && count < 8) {
        parts[count++] = p;
        p = strtok(NULL, "|");
    }

    if (count < 2) {
        printf("Invalid pipeline.\n");
        return;
    }

    int pipes[7][2];
    for (int i = 0; i < count - 1; i++)
        if (pipe(pipes[i]) == -1) {
            perror("pipe");
        }
}
```

```
        return;
    }

    pid_t pids[8];

    for (int i = 0; i < count; i++) {
        char *args[MAX_ARGS];
        int argc = tokenize(parts[i], args);

        if (argc == 0)
            continue;

        pids[i] = fork();

        if (pids[i] == -1) {
            perror("fork");
            return;
        }

        if (pids[i] == 0) {
            signal(SIGINT, SIG_DFL);

            if (i > 0)
                dup2(pipes[i - 1][0], STDIN_FILENO);

            if (i < count - 1)
                dup2(pipes[i][1], STDOUT_FILENO);

            for (int j = 0; j < count - 1; j++) {
                close(pipes[j][0]);
                close(pipes[j][1]);
            }

            execvp(args[0], args);
            perror("execvp");
            _exit(127);
        }
    }

    for (int i = 0; i < count - 1; i++) {
```

```
        close(pipes[i][0]);
        close(pipes[i][1]);
    }

    for (int i = 0; i < count; i++)
        waitpid(pids[i], NULL, 0);

    printf("Pipeline completed successfully.\n");
}

int main(void) {
    char line[1024];

    struct sigaction sa;
    sa.sa_handler = sigint_handler;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;
    sigaction(SIGINT, &sa, NULL);

    printf("=====\n");
    printf(" OSSP FINAL DEMO - MINI UNIX SHELL\n");
    printf("=====\n");
    printf("Supported: normal commands, |, &, jobs, cd, pwd, exit\n\n");

    while (1) {
        update_jobs();

        printf("ossps> ");
        fflush(stdout);

        if (!fgets(line, sizeof(line), stdin))
            break;

        line[strcspn(line, "\n")] = '\0';

        if (line[0] == '\0')
            continue;

        if (strcmp(line, "exit") == 0)
            break;
    }
}
```



sai_pavani@Sai-Pavani: ~/OS_



GNU nano 8.7.1

skill_24.c.c

```
if (strcmp(line, "exit") == 0)
    break;

if (strcmp(line, "pwd") == 0) {
    char cwd[1024];
    if (getcwd(cwd, sizeof(cwd)))
        printf("%s\n", cwd);
    else
        perror("getcwd");
    continue;
}

if (strncmp(line, "cd ", 3) == 0) {
    if (chdir(line + 3) == -1)
        perror("cd");
    continue;
}

if (strcmp(line, "jobs") == 0) {
    for (int i = 0; i < MAX_JOBS; i++)
        if (jobs[i].active)
            printf("[%d] Running pid=%d\n",
                jobs[i].id, jobs[i].pid);
    continue;
}

if (strchr(line, '|')) {
    run_pipeline(line);
    continue;
}

char *args[MAX_ARGS];
int argc = tokenize(line, args);

if (argc == 0)
    continue;

int background = 0;
```

```
        if (strcmp(args[argc - 1], "&") == 0) {
            background = 1;
            args[--argc] = NULL;
        }

        run_simple(args, background);
    }

    // Reap remaining background jobs.
    for (int i = 0; i < MAX_JOBS; i++)
        if (jobs[i].active)
            waitpid(jobs[i].pid, NULL, 0);

    printf("Final demo ended cleanly.\n");
    return 0;
}
```

```

sai_pavani@Sai-Pavani:~/OS_SKILL$ nano skill_24.c.c
sai_pavani@Sai-Pavani:~/OS_SKILL$ gcc skill_24.c.c -o skill_24
sai_pavani@Sai-Pavani:~/OS_SKILL$ ./skill_24
=====
  OSSP FINAL DEMO - MINI UNIX SHELL
=====
Supported: normal commands, |, &, jobs, cd, pwd, exit

ossp> |
Invalid pipeline.
ossp> ls
shellforge          skill3.c            skill_13            skill_18.c.c       skill_24
shellforge.c        skill4              skill_13.c.c        skill_19           skill_24.c.c
simplecommandloop    skill4.c            skill_14            skill_19.c.c       skill_8
simplecommandloop.c skill5              skill_14.c.c        skill_20           skill_8.c.c
skill13_input.txt   skill5.c            skill_15            skill_20.c.c       skill_9
skill13_output.txt  skill_10            skill_15.c.c        skill_21           skill_9.c.c
skill14_append.txt  skill_10.c.c        skill_16            skill_21.c.c       temp
skill14_error.txt   skill_11            skill_16.c.c        skill_22
skill15_combined.txt skill_11.c.c        skill_17            skill_22.c.c
skill21_errors.log  skill_12            skill_17.c.c        skill_23
skill3              skill_12.c.c        skill_18            skill_23.c.c
ossp> pwd
/home/sai_pavani/OS_SKILL
ossp> cd ..
ossp> pwd
/home/sai_pavani
ossp> cd OS_SKILL
ossp> echo Hello | wc -c
6
Pipeline completed successfully.
ossp> ls | wc -l
51
Pipeline completed successfully.
ossp> sleep 5 &
[1] 1891
ossp> jobs
[1] Running pid=1891
[1] Done
ossp> jobs
ossp> sleep 10
ossp> exit
Final demo ended cleanly.
sai_pavani@Sai-Pavani:~/OS_SKILL$ |

```